Chiến lược và Kế hoạch Triển khai Hệ thống Local Data Lakehouse: Tích hợp Apache Spark, Iceberg, Project Nessie và Apache Airflow trên Nền tảng MinIO

Sự chuyển dịch từ các kho dữ liệu (Data Warehouse) truyền thống sang các hồ dữ liệu (Data Lake) và hiện nay là sự kết hợp trong mô hình Data Lakehouse đã đánh dấu một bước ngoặt quan trọng trong kỷ nghệ dữ liệu hiện đại. Mô hình Data Lakehouse hướng tới việc cung cấp khả năng quản lý dữ liệu có cấu trúc với tính toàn vẹn giao dịch của kho dữ liệu, đồng thời duy trì tính linh hoạt và khả năng mở rộng của hồ dữ liệu.[1] Trong bối cảnh triển khai cục bộ (local deployment), việc kết hợp Apache Spark làm động cơ xử lý, Apache Iceberg làm định dạng bảng, Project Nessie làm catalog quản lý phiên bản và Apache Airflow làm nền tảng điều phối, trên lớp lưu trữ MinIO, tạo ra một hệ sinh thái mạnh mẽ để xử lý các tập dữ liệu quy mô lớn như dữ liệu Taxi New York (NYC Taxi).[1]

Phân tích Kiến trúc Hệ thống Lakehouse Cục bộ

Kiến trúc Lakehouse được thiết kế nhằm giải quyết các thách thức cố hữu của các hồ dữ liệu truyền thống, vốn thường gặp vấn đề về chất lượng dữ liệu và khó khăn trong việc thực hiện các giao dịch ACID.[2] Hệ thống này được xây dựng dựa trên bốn lớp chức năng cốt lõi, mỗi lớp đảm nhiệm một vai trò chuyên biệt để đảm bảo tính nhất quán và hiệu suất của luồng dữ liệu.

Lớp Lưu trữ Đối tượng: MinIO

MinIO đóng vai trò là lớp lưu trữ vật lý, cung cấp giao diện tương thích hoàn toàn với API Amazon S3. Trong môi trường local, MinIO cho phép lưu trữ cả dữ liệu thô (raw data) và các tệp dữ liệu định dạng Parquet được quản lý bởi Iceberg.[1] Việc sử dụng MinIO mang lại lợi ích về tính di động cao, cho phép các cấu hình và mã nguồn phát triển cục bộ có thể chuyển đổi sang môi trường đám mây thực tế như AWS S3 mà không cần thay đổi lớn về kiến trúc.[7]

Lớp Định dạng Bảng: Apache Iceberg

Apache Iceberg là trung tâm của kiến trúc Lakehouse, cung cấp các tính năng quản trị dữ liệu ở cấp độ bảng. Khác với cách tiếp cận dựa trên thư mục của Hive, Iceberg sử dụng cấu trúc phân cấp siêu dữ liệu (metadata) để theo dõi các snapshot của bảng.[2] Các đặc tính then chốt của Iceberg bao gồm:

- **Giao dịch ACID:** Đảm bảo tính nguyên tử (Atomicity), nhất quán (Consistency), cô lập (Isolation) và bền vững (Durability) cho mọi thao tác ghi và đọc dữ liệu.[2]
- **Tiến hóa Lược đồ (Schema Evolution):** Cho phép thay đổi cấu trúc bảng (thêm, xóa, đổi

tên cột) mà không cần ghi lại toàn bộ dữ liệu.[9]
- **Tiến hóa Phân vùng (Partition Evolution):** Cho phép thay đổi chiến lược phân vùng theo thời gian khi khối lượng dữ liệu thay đổi mà không làm hỏng các truy vấn hiện có.[5]

Lớp Catalog và Quản lý Phiên bản: Project Nessie

Project Nessie giới thiệu khái niệm "Git cho dữ liệu" vào hệ thống Lakehouse. Trong khi Iceberg quản lý trạng thái của từng bảng riêng lẻ, Nessie quản lý trạng thái của toàn bộ catalog.[1] Nessie cho phép các kỹ sư dữ liệu tạo ra các nhánh (branches) để thực hiện các quy trình ETL thử nghiệm, thực hiện các thay đổi một cách cô lập và chỉ hợp nhất (merge) vào nhánh chính (main branch) sau khi dữ liệu đã được kiểm định chất lượng.[13]

Lớp Xử lý và Điều phối: Spark và Airflow

Apache Spark là động cơ tính toán phân tán chính, chịu trách nhiệm cho các tác vụ ETL phức tạp. Khi tích hợp với Iceberg, Spark đóng vai trò là engine thực thi các lệnh SQL hoặc các thao tác DataFrame, tương tác trực tiếp với metadata của Iceberg để thực hiện các thay đổi dữ liệu.[1] Apache Airflow đóng vai trò là "bộ não" điều phối, tự động hóa việc lập lịch và giám sát các quy trình từ lúc ingestion dữ liệu thô đến khi tạo ra các bảng tổng hợp cuối cùng.[1]

Thiết lập Hạ tầng Kỹ thuật với Docker Compose

Để triển khai một hệ thống Lakehouse cục bộ, việc sử dụng Docker Compose là phương pháp tối ưu nhằm đảm bảo tính đồng nhất của môi trường và quản lý dễ dàng các phụ thuộc giữa các dịch vụ.[1]

Cấu hình các Dịch vụ Cốt lõi

Hệ thống yêu cầu sự phối hợp giữa nhiều dịch vụ container. Dưới đây là bảng chi tiết các dịch vụ cần thiết và vai trò của chúng:

Dịch vụ	Hình ảnh (Image) Gợi ý	Vai trò	Cổng (Port)
MinIO	minio/minio	Lưu trữ đối tượng S3-compatible [7]	9000 (API), 9001 (UI)
Nessie	ghcr.io/projectnessie/nessie	Catalog quản lý phiên bản dữ liệu [14]	19120
Spark Master	bitnami/spark:3.5	Điều phối cụm	7077, 8080 (UI)

		Spark [21]	
Spark Worker	bitnami/spark:3.5	Thực thi các tác vụ xử lý dữ liệu [21]	8081 (UI)
Airflow	apache/airflow:2.10.x	Điều phối quy trình (DAGs) [23]	8080 (UI)
PostgreSQL	postgres:13	Lưu trữ metadata cho Airflow [25]	5432

Quản lý Kết nối và Quyền truy cập

Một trong những thách thức lớn nhất khi chạy Spark và Airflow trong Docker là việc quản lý quyền truy cập tệp và khả năng hiển thị logs. Các chuyên gia khuyến nghị tạo một volume dùng chung có tên spark-events gắn vào thư mục ${SPARK_HOME}/events trên tất cả các container liên quan đến Spark và Airflow.[23] Điều này cho phép Spark History Server và Airflow Webserver có thể truy cập vào các tệp log sự kiện để hiển thị trạng thái thực thi công việc. Ngoài ra, việc sử dụng dịch vụ airflow-init để chạy lệnh chown -R là cần thiết để đảm bảo user airflow (thường là UID 50000) có quyền ghi vào các thư mục được ánh xạ từ máy chủ local.[23]

Chiến lược Xử lý Dữ liệu theo Mô hình Medallion

Dữ liệu NYC Taxi là một tập dữ liệu điển hình cho việc áp dụng kiến trúc Medallion (Bronze, Silver, Gold), giúp cải thiện chất lượng dữ liệu qua từng giai đoạn xử lý.[27]

Định nghĩa các Lớp Dữ liệu

Tầng (Layer)	Mục tiêu	Đặc điểm Xử lý
Bronze	Lưu trữ nguyên bản	Ingest dữ liệu nyc_taxi_data.parquet vào bảng Iceberg mà không thay đổi cấu trúc. Giữ lại dấu vết kiểm toán (audit trail).[27]
Silver	Làm sạch và Làm giàu	Loại bỏ các bản ghi không hợp lệ (ví dụ: fare_amount < 0), chuẩn hóa kiểu dữ liệu, và tính toán thêm các cột

		mới như trip_duration_minutes.[16]
Gold	Tổng hợp kinh doanh	Tạo các bảng tóm tắt phục vụ báo cáo, ví dụ: doanh thu trung bình theo giờ, số lượng chuyến đi theo khu vực.[28]

Chi tiết Lược đồ Dữ liệu NYC Taxi

Dữ liệu NYC Taxi chứa các thông tin chi tiết về các chuyến đi, đòi hỏi việc xử lý kiểu dữ liệu và giá trị thiếu một cách cẩn thận.[16]

Tên Cột	**Kiểu Dữ liệu**	**Ý nghĩa**
VendorID	Integer	Mã định danh nhà cung cấp dịch vụ [16]
tpep_pickup_datetime	Timestamp	Thời điểm đón khách [16]
passenger_count	Double/Integer	Số lượng hành khách trên chuyến đi [16]
trip_distance	Double	Khoảng cách chuyến đi (miles) [16]
fare_amount	Double	Số tiền cước vận chuyển [16]
total_amount	Double	Tổng chi phí (bao gồm thuế, phí, tips) [35]
PULocationID	Integer	Mã vùng đón khách [34]

Cấu hình Chuyên sâu cho Apache Spark với Iceberg và Nessie

Để Spark có thể giao tiếp với Iceberg và catalog Nessie trên nền tảng MinIO, các thuộc tính cấu hình trong spark-defaults.conf phải được thiết lập một cách chính xác.[36]

Các Phụ thuộc JAR Cần thiết

Người dùng cần đảm bảo các thư viện sau có mặt trong classpath của Spark (thường thông qua tham số --packages hoặc đặt trong thư mục jars/):

- org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:1.6.0 (hoặc phiên bản mới nhất tương thích).[39]
- org.projectnessie.nessie-integrations:nessie-spark-extensions-3.5_2.12:0.99.0 (để hỗ trợ cú pháp SQL mở rộng của Nessie).[39]
- org.apache.hadoop:hadoop-aws:3.3.x và com.amazonaws:aws-java-sdk-bundle:1.12.x (để kết nối với S3/MinIO).[7]

Cấu hình Hệ thống (spark-defaults.conf)

Properties

```
# Kích hoạt Iceberg Spark Extensions
spark.sql.extensions = org.apache.iceberg.spark.extensions.IcebergSparkSessionExtensions,org.projectnessie.spark.extensions.NessieSparkSessionExtensions

# Cấu hình Catalog Nessie
spark.sql.catalog.nessie = org.apache.iceberg.spark.SparkCatalog
spark.sql.catalog.nessie.catalog-impl = org.apache.iceberg.nessie.NessieCatalog
spark.sql.catalog.nessie.uri = http://nessie:19120/api/v1
spark.sql.catalog.nessie.ref = main
spark.sql.catalog.nessie.warehouse = s3a://warehouse/
spark.sql.catalog.nessie.authentication.type = NONE

# Cấu hình Kết nối MinIO (S3A)
spark.hadoop.fs.s3a.endpoint = http://minio:9000
spark.hadoop.fs.s3a.access.key = admin
spark.hadoop.fs.s3a.secret.key = password
spark.hadoop.fs.s3a.path.style.access = true
spark.hadoop.fs.s3a.impl = org.apache.hadoop.fs.s3a.S3AFileSystem
spark.hadoop.fs.s3a.connection.ssl.enabled = false
```

6

Việc sử dụng giao thức s3a là bắt buộc để Spark có thể tương tác với các hệ thống lưu trữ tương thích S3 thông qua lớp Hadoop FileSystem.[6] Ngoài ra, thuộc tính path.style.access đặt là

true rất quan trọng đối với MinIO để tránh việc phân giải DNS theo kiểu bucket-name.domain.[6]

Quản lý Phiên bản Dữ liệu: Quy trình làm việc với Nessie

Một trong những ưu điểm vượt trội của Nessie là khả năng triển khai mô hình "Write-Audit-Publish" (WAP) một cách tự nhiên thông qua các nhánh dữ liệu.[12]

Quy trình Phân nhánh và Hợp nhất

Quy trình làm việc chuẩn cho một tác vụ ETL taxi hàng tháng thường bao gồm:

1. **Tạo Nhánh Mới:** Trước khi bắt đầu ghi dữ liệu, một nhánh cô lập được tạo ra (ví dụ: etl_jan_2024).
   - SQL: CREATE BRANCH etl_jan_2024 IN nessie.[13]
2. **Ghi Dữ liệu Cô lập:** Spark thực hiện ghi dữ liệu vào các bảng Iceberg nhưng chỉ định đích là nhánh vừa tạo. Điều này đảm bảo rằng các truy vấn hiện tại trên nhánh main không bị ảnh hưởng bởi dữ liệu đang được xử lý hoặc dữ liệu rác nếu quá trình ETL thất bại.[13]
   - SQL: USE REFERENCE etl_jan_2024 IN nessie.[14]
3. **Kiểm định Chất lượng (Auditing):** Các truy vấn kiểm tra (data quality checks) được thực thi trên nhánh etl_jan_2024. Nếu dữ liệu không vượt qua các bài kiểm tra này, nhánh có thể bị xóa hoặc sửa chữa mà không để lại tác động xấu nào đến production.[13]
4. **Hợp nhất vào Nhánh Chính:** Sau khi xác nhận dữ liệu đã "sạch" và chính xác, toàn bộ các thay đổi trên nhiều bảng được hợp nhất vào nhánh main như một giao dịch nguyên tử duy nhất.[13]
   - SQL: MERGE BRANCH etl_jan_2024 INTO main IN nessie.[13]

Khả năng Du hành Thời gian (Time Travel)

Iceberg và Nessie cung cấp khả năng truy vấn dữ liệu tại một thời điểm hoặc snapshot cụ thể. Điều này cực kỳ hữu ích cho việc tái lập các báo cáo cũ hoặc so sánh hiệu quả của các mô hình học máy trên cùng một phiên bản dữ liệu.[5]

- Truy vấn theo timestamp: SELECT * FROM taxi.silver AT '2024-01-01 10:00:00'.[3]
- Truy vấn theo snapshot ID: SELECT * FROM taxi.silver AT SNAPSHOT '123456789'.[3]

Điều phối Quy trình ETL với Apache Airflow

Trong môi trường Lakehouse, Airflow không thực hiện tính toán mà đóng vai trò điều khiển và giám sát các tác vụ Spark.[24]

Cấu hình Kết nối Spark trong Airflow

Để Airflow có thể gửi công việc đến cụm Spark, người dùng cần tạo một Connection trong

Airflow UI (Admin -> Connections) [23]:

- **Conn Id:** spark_default
- **Conn Type:** Spark
- **Host:** spark://spark-master
- **Port:** 7077
- **Deploy Mode:** client (trong môi trường Docker, mode client giúp driver chạy ngay trong worker của Airflow, giúp dễ dàng quản lý logs và volumes).[23]

Sử dụng SparkSubmitOperator

SparkSubmitOperator là công cụ chính để gửi các tập lệnh PySpark. Việc đóng gói logic ETL vào các tệp .py riêng biệt giúp mã nguồn dễ bảo trì và kiểm thử hơn.[18]

Python

```python
from airflow.providers.apache.spark.operators.spark_submit import SparkSubmitOperator

task_ingest_bronze = SparkSubmitOperator(
    task_id="ingest_nyc_taxi_to_bronze",
    application="/opt/airflow/jobs/ingest_bronze.py",
    conn_id="spark_default",
    conf={
        "spark.sql.catalog.nessie.ref": "etl_branch_name",
        "spark.jars.packages": "org.apache.iceberg:iceberg-spark-runtime-3.5_2.12:1.6.0,..."
    },
    verbose=True
)
```

18

Một lưu ý quan trọng là việc sử dụng PythonOperator để chạy logic ETL nặng bằng thư viện pyspark trực tiếp bên trong Airflow worker là một hành vi không được khuyến khích (bad practice) vì nó sẽ chiếm dụng tài nguyên của hệ thống điều phối và không có khả năng mở rộng (scale out).[49]

Vận hành và Tối ưu hóa Bảng Iceberg

Việc duy trì hiệu suất cho Lakehouse đòi hỏi các hoạt động bảo trì bảng định kỳ, vì Iceberg lưu trữ dữ liệu dưới dạng các tệp bất biến.[5]

Các Tác vụ Bảo trì Cần thiết

1. **Nén Tệp (Compaction):** Khi có nhiều hoạt động ghi nhỏ, số lượng tệp Parquet sẽ tăng lên, làm chậm quá trình truy vấn. Việc nén các tệp nhỏ thành các tệp lớn hơn giúp tối ưu hóa I/O.[5]
   - Lệnh Spark: CALL nessie.system.rewrite_data_files('db.table').
2. **Xóa Snapshot Cũ (Expire Snapshots):** Để tiết kiệm không gian lưu trữ và giữ cho metadata gọn nhẹ, các snapshot cũ không còn cần thiết cho việc rollback nên được xóa bỏ.[5]
   - Lệnh Spark: CALL nessie.system.expire_snapshots('db.table', timestamp).
3. **Dọn dẹp Tệp Mồ côi (Remove Orphan Files):** Xóa các tệp vật lý trên MinIO mà không còn được tham chiếu bởi bất kỳ snapshot nào trong metadata.[5]

Tối ưu hóa Truy vấn thông qua Phân vùng và Chỉ mục

Sử dụng "Hidden Partitioning" của Iceberg để phân vùng dữ liệu taxi theo ngày đón khách (pickup_date). Điều này cho phép Spark thực hiện "Partition Pruning", chỉ đọc các thư mục dữ liệu cần thiết thay vì quét toàn bộ bảng.[5]

Lộ trình Triển khai Chi tiết (Coding Roadmap)

Để bắt tay vào viết code, dự án nên được chia thành các giai đoạn nhỏ để dễ dàng kiểm soát lỗi và đảm bảo từng thành phần hoạt động đúng.

Giai đoạn 1: Khởi tạo Hạ tầng Docker

- Xây dựng tệp docker-compose.yaml bao gồm MinIO, Nessie, Spark và Airflow.[1]
- Tạo Custom Dockerfile cho Airflow để cài đặt thêm apache-airflow-providers-apache-spark và JDK.[23]
- Thiết lập mạng Docker chung để các container có thể giao tiếp qua hostname.[21]
- Khởi động hệ thống: docker-compose up -d.

Giai đoạn 2: Cấu hình Catalog và Lưu trữ

- Truy cập MinIO Console (cổng 9001) để tạo bucket warehouse.[3]
- Tạo Access Key và Secret Key cho MinIO và lưu vào tệp .env.[3]
- Viết tệp spark-defaults.conf với đầy đủ các cấu hình catalog Nessie và endpoint MinIO.[7]

Giai đoạn 3: Phát triển Luồng Ingestion (Bronze Layer)

- Tải dữ liệu mẫu nyc_taxi_data.parquet vào thư mục local được ánh xạ vào container.[33]
- Viết script PySpark để đọc tệp này và ghi vào bảng Iceberg nessie.taxi.bronze.[16]
- Sử dụng Nessie CLI hoặc SQL để kiểm tra commit đầu tiên trong catalog.[12]

Giai đoạn 4: Biến đổi Dữ liệu (Silver Layer)

- Xây dựng script làm sạch dữ liệu: lọc tọa độ không hợp lệ, chuẩn hóa tiền tệ, xử lý null cho passenger_count.[16]
- Tính toán các cột phái sinh: duration = dropoff_time - pickup_time.[28]
- Ghi dữ liệu vào nessie.taxi.silver với chiến lược phân vùng theo ngày.[28]

Giai đoạn 5: Tổng hợp và Báo cáo (Gold Layer)

- Viết các câu lệnh SQL để tạo bảng tổng hợp: revenue_by_location, avg_trip_distance_by_passenger_count.[28]
- Lưu trữ kết quả vào nessie.taxi.gold.[28]

Giai đoạn 6: Tự động hóa với Airflow

- Viết DAG chính để liên kết các bước trên.[18]
- Tích hợp logic quản lý nhánh của Nessie: Tạo nhánh -> Chạy ETL -> Kiểm tra -> Merge.[13]
- Thiết lập lịch chạy (schedule) hàng ngày hoặc hàng tuần.

Phân tích Chuyên sâu về Hiệu quả của Giải pháp

Việc lựa chọn Spark phối hợp với Iceberg và Nessie thay vì các engine khác như Trino mang lại lợi thế lớn cho các tác vụ batch xử lý khối lượng lớn dữ liệu (Heavy ETL).[1] Spark có khả năng xử lý lỗi tốt hơn thông qua cơ chế checkpoint và khả năng quản lý bộ nhớ linh hoạt đối với các phép join lớn.

So sánh Hiệu suất Lưu trữ và Truy vấn

Sử dụng định dạng Parquet trong Iceberg giúp giảm đáng kể dung lượng lưu trữ so với CSV truyền thống nhờ kỹ thuật nén theo cột.[51] Khi kết hợp với Metadata của Iceberg, tốc độ truy vấn các tập dữ liệu hàng tỷ dòng (như toàn bộ lịch sử NYC Taxi) được cải thiện nhờ khả năng loại bỏ (pruning) các tệp không liên quan ở mức độ tinh vi hơn nhiều so với Hive.[2]

Ý nghĩa của Tính năng Phân nhánh Dữ liệu

Tính năng phân nhánh của Nessie không chỉ là một công cụ kỹ thuật mà còn là một cơ chế quản trị dữ liệu (data governance). Nó cho phép triển khai môi trường "Multi-tenant" trên cùng một storage layer. Ví dụ, một nhóm Data Science có thể tạo nhánh ml_experiment từ nhánh main để thực hiện feature engineering mà không làm xáo trộn dữ liệu đang phục vụ cho báo cáo tài chính trên nhánh main.[4] Điều này loại bỏ hoàn toàn việc phải sao chép dữ liệu vật lý (data duplication), giúp tiết kiệm chi phí lưu trữ đáng kể.

Các Thách thức và Giải pháp Khắc phục

Mặc dù kiến trúc này rất mạnh mẽ, việc triển khai cục bộ có thể gặp một số vấn đề về tài nguyên phần cứng.

- **Giới hạn RAM:** Chạy Airflow, Spark và MinIO đồng thời yêu cầu tối thiểu 8GB-16GB RAM.[20] Người dùng nên giới hạn số lượng executor của Spark và điều chỉnh tham số memory trong Docker Compose.
- **Xung đột Phiên bản:** Việc lựa chọn sai phiên bản JAR giữa Iceberg, Nessie và Spark là nguyên nhân hàng đầu gây ra lỗi.[17] Luôn kiểm tra bảng ma trận tương thích (Compatibility Matrix) trên trang chủ của Project Nessie và Apache Iceberg trước khi cấu hình.[22]

Kết luận và Khuyến nghị

Việc xây dựng một Local Data Lakehouse với Spark, Iceberg, Nessie và Airflow là một bước chuẩn bị xuất sắc cho bất kỳ dự án dữ liệu lớn nào. Kiến trúc này cung cấp đầy đủ các tính năng hiện đại nhất của một nền tảng dữ liệu doanh nghiệp: tính toàn vẹn ACID, quản lý phiên bản kiểu Git và khả năng điều phối tự động.[1]

Đối với dữ liệu NYC Taxi, việc áp dụng mô hình Medallion giúp chuẩn hóa quy trình từ lúc tiếp nhận dữ liệu thô cho đến khi cung cấp thông tin chi tiết cho doanh nghiệp.[28] Kế hoạch triển khai sáu giai đoạn được đề xuất cung cấp một lộ trình rõ ràng, giúp người phát triển có thể xây dựng hệ thống một cách có hệ thống và bền vững. Sau khi hoàn thành việc triển khai cục bộ, hệ thống này có thể dễ dàng được mở rộng lên các nền tảng đám mây như AWS hoặc Kubernetes với sự thay đổi tối thiểu về mặt logic mã nguồn.[6] Đây chính là nền tảng vững chắc để phát triển các ứng dụng phân tích nâng cao và học máy trong tương lai.

Works cited

1. vutrinh274/local_lakehouse - GitHub, accessed February 24, 2026, https://github.com/vutrinh274/local_lakehouse
2. What Is Apache Iceberg? - IBM, accessed February 24, 2026, https://www.ibm.com/think/topics/apache-iceberg
3. domainio/iceberglakehouse: Building a Data Lakehouse with Apache Iceberg, Spark, Dremio, Nessie & Minio - GitHub, accessed February 24, 2026, https://github.com/domainio/iceberglakehouse
4. Nessie Catalog | Ilum - Apache Spark on Kubernetes Platform | Documentation, accessed February 24, 2026, https://ilum.cloud/docs/features/catalogs/nessie/
5. Iceberg Tables Management: Processes, Challenges & Best Practices - lakeFS, accessed February 24, 2026, https://lakefs.io/blog/iceberg-tables-management/
6. Integrating MinIO with Apache Spark for Big Data Processing - DoHost, accessed February 24, 2026, https://dohost.us/index.php/2025/11/14/integrating-minio-with-apache-spark-for-big-data-processing/
7. Setting up AWS S3 (Minio) and Spark using Docker Compose | by Dimitris Kalouris, accessed February 24, 2026,

https://medium.com/@dkalouris/setting-up-aws-s3-minio-and-spark-using-docker-compose-6a22ef26c6b0
8. Setting Up Spark Standalone with MinIO Object Storage Using Docker Compose - Medium, accessed February 24, 2026, https://medium.com/@mklmfernando/setting-up-spark-standalone-with-minio-object-storage-using-docker-compose-c0fdaebbace9
9. Create Data Lakehouse Using Spark+Iceberg+Nessie+Dremio | by Burak Uğur - Medium, accessed February 24, 2026, https://medium.com/towards-data-engineering/create-data-lakehouse-using-spark-iceberg-nessie-dremio-8d7a54855f13
10. Handling Schema Drift in Medallion Architecture with Apache Iceberg - Nexla, accessed February 24, 2026, https://nexla.com/blog/handling-schema-drift-in-medallion-architecture-with-apache-iceberg/
11. Tutorial: Intro to Apache Iceberg with Apache Polaris and Apache Spark - DEV Community, accessed February 24, 2026, https://dev.to/alexmercedcoder/tutorial-intro-to-apache-iceberg-with-apache-polaris-and-apache-spark-p70
12. Nessie Catalog: Key Features, Use Cases & How to Use - lakeFS, accessed February 24, 2026, https://lakefs.io/blog/nessie-catalog/
13. Getting started with Nessie, Iceberg and Spark using Docker - Dremio, accessed February 24, 2026, https://www.dremio.com/getting-started-with-project-nessie-apache-iceberg-and-apache-spark-using-docker/
14. nessie-demos/notebooks/nessie-iceberg-demo-nba.ipynb at main · projectnessie/nessie-demos - GitHub, accessed February 24, 2026, https://github.com/projectnessie/nessie-demos/blob/main/notebooks/nessie-iceberg-demo-nba.ipynb
15. About Nessie - Project Nessie: Transactional Catalog for Data Lakes with Git-like semantics, accessed February 24, 2026, https://projectnessie.org/guides/about/
16. Analyze data with Apache Spark and Python - Microsoft Fabric, accessed February 24, 2026, https://learn.microsoft.com/en-us/fabric/data-science/python-guide/python-visualizations
17. How to query Iceberg locally using Spark, PyIceberg, or duckdb - Fivetran, accessed February 24, 2026, https://www.fivetran.com/blog/how-to-query-iceberg-locally-using-spark-pyiceberg-or-duckdb
18. Apache Airflow SparkSubmitOperator: A Comprehensive Guide - SparkCodeHub, accessed February 24, 2026, https://www.sparkcodehub.com/airflow/operators/spark-submit-operator
19. The Definitive Guide to Lakehouse Architecture with Iceberg and ..., accessed February 24, 2026, https://blog.min.io/the-definitive-guide-to-lakehouse-architecture-with-iceberg-and-aistor/

20. Running Airflow in Docker — Airflow 3.1.7 Documentation, accessed February 24, 2026, https://airflow.apache.org/docs/apache-airflow/stable/howto/docker-compose/index.html
21. Building a Production-Ready ETL Pipeline with Docker Compose: Airflow 3.0, Spark, and Microservices - Daniel Egbo, accessed February 24, 2026, https://danielegbo.medium.com/building-a-production-ready-etl-pipeline-with-docker-compose-airflow-spark-and-microservices-c2f12cb6d85c
22. projectnessie/nessie: Nessie: Transactional Catalog for Data Lakes with Git-like semantics - GitHub, accessed February 24, 2026, https://github.com/projectnessie/nessie
23. Setting up Spark and Airflow using Docker Compose | by Dimitris Kalouris | Medium, accessed February 24, 2026, https://medium.com/@dkalouris/setting-up-and-connecting-airflow-and-spark-using-docker-compose-9773dec21bc8
24. Spark, Iceberg and Airflow Quickstart | by ahmed trifa | Medium, accessed February 24, 2026, https://medium.com/@ahmed.trifa1/spark-iceberg-and-airflow-quickstart-369956327fe0
25. Apache Spark & Airflow in Docker: Step by Step guide | by Sanjeet Shukla | Dev Genius, accessed February 24, 2026, https://blog.devgenius.io/apache-spark-airflow-in-docker-step-by-step-guide-fe02bcdcdedc
26. Connecting to master spark://spark-master:7077 All masters are unresponsive! Giving up, accessed February 24, 2026, https://stackoverflow.com/questions/77792866/connecting-to-master-spark-spark-master7077-all-masters-are-unresponsive-giv
27. What is a Medallion Architecture? - Databricks, accessed February 24, 2026, https://www.databricks.com/glossary/medallion-architecture
28. Phase 2: Building a Bronze → Silver → Gold ETL Pipeline with PySpark for NYC Taxi Streaming & Analytics Project | by Hamza Paracha | Medium, accessed February 24, 2026, https://medium.com/@hamzaparacha098/phase-2-building-a-bronze-silver-gold-etl-pipeline-with-pyspark-for-nyc-taxi-streaming-44e970f357b4
29. himanshuTaleleNeu/nyc_taxi_data_engineering: End-to-End Data Pipeline for NYC Green Taxi trip data using Azure Data Factory, Data Lake, and Databricks following the Medallion architecture - GitHub, accessed February 24, 2026, https://github.com/himanshuTaleleNeu/nyc_taxi_data_engineering
30. Bronze, Silver, Gold: Build Reliable Data Pipelines with Medallion Architecture by Databricks | by Harisudhan.S | Medium, accessed February 24, 2026, https://medium.com/@speaktoharisudhan/bronze-silver-gold-build-reliable-data-pipelines-with-medallion-architecture-by-databricks-ce17201b7bee
31. Building the Medallion Architecture with Delta Lake, accessed February 24, 2026, https://delta.io/blog/delta-lake-medallion-architecture/
32. Gold Layer PySpark ETL With AWS Glue Studio - amazonwebshark, accessed

February 24, 2026, https://amazonwebshark.com/gold-layer-pyspark-etl-with-aws-glue-studio/
33. How I Built a Complete NYC Taxi Data Pipeline Using SQL | by Raphael Patrick-Affiah, accessed February 24, 2026, https://medium.com/@patraffiah/how-i-built-a-complete-nyc-taxi-data-pipeline-using-sql-dbe84ce23192
34. Uber NYC TLC data | Exploratory Data Analysis - Kaggle, accessed February 24, 2026, https://www.kaggle.com/code/shuhengmo/uber-nyc-tlc-data-exploratory-data-analysis
35. Explore public datasets with Apache Iceberg & BigLake | Google Open Source Blog, accessed February 24, 2026, https://opensource.googleblog.com/2026/01/explore-public-datasets-with-apache-iceberg-and-biglake.html
36. Spark Configuration - Apache Iceberg, accessed February 24, 2026, https://iceberg.apache.org/docs/latest/spark-configuration/
37. Spark Configuration - Apache Iceberg, accessed February 24, 2026, https://iceberg.apache.org/docs/1.5.0/spark-configuration/
38. Nessie - Apache Iceberg™, accessed February 24, 2026, https://iceberg.apache.org/docs/1.4.3/nessie/
39. Nessie + Iceberg + Spark - Project Nessie: Transactional Catalog for Data Lakes with Git-like semantics, accessed February 24, 2026, https://projectnessie.org/iceberg/spark/
40. Nessie Spark SQL Extensions - Project Nessie: Transactional Catalog for Data Lakes with Git-like semantics, accessed February 24, 2026, https://projectnessie.org/guides/sql/
41. iceberg-spark-3.5_2.12 » 1.6.0 - Maven Repository, accessed February 24, 2026, https://mvnrepository.com/artifact/org.apache.iceberg/iceberg-spark-3.5_2.12/1.6.0
42. Nessie - Apache Iceberg™, accessed February 24, 2026, https://iceberg.apache.org/docs/1.9.2/nessie/
43. ucesys/DataLakehouse - GitHub, accessed February 24, 2026, https://github.com/ucesys/DataLakehouse
44. Local Spark With MinIO S3 emulation, and Hive Metastore - John Gallagher, accessed February 24, 2026, http://slackhappy.github.io/local_spark_s3.html
45. abeltavares/versioned-data-lakehouse: Git-like Version ... - GitHub, accessed February 24, 2026, https://github.com/abeltavares/versioned-data-lakehouse
46. Setting up Apache Iceberg locally using PySpark | Engineering | ClickHouse Resource Hub, accessed February 24, 2026, https://clickhouse.com/resources/engineering/apache-iceberg-local-pyspark
47. Apache Airflow or Spark? : r/dataengineering - Reddit, accessed February 24, 2026, https://www.reddit.com/r/dataengineering/comments/1d2tl1t/apache_airflow_or_spark/
48. Apache Spark Operators — apache-airflow-providers-apache-spark

Documentation, accessed February 24, 2026, https://airflow.apache.org/docs/apache-airflow-providers-apache-spark/stable/operators.html
49. So,it's me or Airflow is kinda really hard ? : r/dataengineering - Reddit, accessed February 24, 2026, https://www.reddit.com/r/dataengineering/comments/1nnzk04/soits_me_or_airflow_is_kinda_really_hard/
50. How to Load Data into Apache Iceberg: A Step-by-Step Tutorial - Estuary, accessed February 24, 2026, https://estuary.dev/blog/loading-data-into-apache-iceberg/
51. Use this if you want to convert the nyc-taxi-data into a parquet format for use in Apache Spark - GitHub, accessed February 24, 2026, https://github.com/nwstephens/nyc-taxi-data
52. Dashboard Week Day 3 - Taxi Rides (and working with Parquet files in Alteryx), accessed February 24, 2026, https://www.thedataschool.co.uk/luke-bennett/dashboard-week-day-3-taxi-rides-and-working-with-parquet/
53. Example: NYC taxi data, accessed February 24, 2026, http://download.nust.na/pub3/cran/web/packages/arrow/vignettes/dataset.html